TIENDA TP

Informe de avance del proyecto

Fecha: 15 de octubre de 2026
Materia: Algoritmos y Estructura de Datos III
Alcance: análisis de la consigna, correcciones iniciales, servidor HTTP y persistencia.

1. Estado alcanzado

El proyecto cuenta con un punto de entrada que inicia un servidor HTTP nativo, operaciones de persistencia de productos y herramientas para cargar datos de prueba. Se corrigió el módulo de consultas de clientes y productos. La tienda todavía no integra los TDA con una API ni dispone de una interfaz funcional.

2. Revisión inicial

Se revisaron main.py, validaciones.py, los módulos de BACKEND y los scripts de BD. Se identificaron TDA de lista enlazada, pila, cola y árbol binario; una base SQLite; un frontend sin interfaz; errores de imports, consultas y rutas; y ejecución de SQL al importar algunos scripts.

También se leyó el PDF de consignas completo, de cuatro páginas. El trabajo requiere inventario con lista enlazada, esperas con cola, historial y reversión de compras con pila, y recomendaciones con un BST ordenado por popularidad.

3. Restricciones de la consigna

La aplicación debe usar Python estándar en el backend y HTML5, CSS3 y JavaScript Vanilla con fetch en el frontend. Debe mantener tres capas, persistencia duradera, nombre y logotipo visibles, diseño responsivo, nombres en español y documentación. Los TDA deben ser los de clase y utilizarse mediante sus primitivas, sin accesos indebidos a punteros desde la lógica de negocio.

Condición de trabajo: se mantuvo la instrucción de no realizar operaciones sobre la carpeta .git. Este informe no requiere comandos de Git ni cambios en la base de datos.

Cambios realizados

4. Arranque y servidor HTTP

main.py: se reemplazaron los imports de TDA que no apuntaban a BACKEND por la importación de iniciar_servidor. Se incorporó la guarda __name__ == "__main__" y un comentario que explica el arranque. Importarlo no inicia el servidor.

BACKEND/servidor.py: se implementó iniciar_servidor(host="127.0.0.1", puerto=8000), usando ThreadingHTTPServer y SimpleHTTPRequestHandler. La carpeta FRONTEND se resuelve desde la ubicación del archivo. El servidor informa su dirección, atiende solicitudes y cierra al interrumpirse con Ctrl+C.

Límite actual: sirve archivos estáticos. No tiene rutas de API, conexión del inventario ni procesamiento de compras.

5. Persistencia de productos

BD/persistencia.py: se proporcionó el código y se verificó posteriormente su incorporación al proyecto. La ruta de tablas.db se construye con Path(__file__), las conexiones activan claves foráneas y sqlite3.Row permite convertir los resultados en diccionarios.

Las operaciones disponibles son conectar(), leer_productos(), consultar_producto(), insertar_producto(), modificar_producto() y eliminar_producto(). Las escrituras usan parámetros SQL y un contexto de transacción; todas las funciones de consulta o escritura cierran su conexión con finally.

Se corrigió el uso de cantidad por cantidad_disponible. Los cambios se confirman cuando la operación termina correctamente y se revierten ante una excepción dentro del contexto de escritura.

Alcance: la validación de negocio y el manejo de errores en la futura API siguen pendientes. Las claves foráneas pueden impedir eliminar productos relacionados con compras o esperas. No se implementó un CRUD completo de clientes, por decisión del usuario.

6. Separación de responsabilidades

Se explicó la separación entre crear el esquema, cargar ejemplos y realizar operaciones habituales. Se propuso crear_tablas.py con CREATE TABLE IF NOT EXISTS, pero no se creó en esta sesión porque las tablas ya existían. BD/app.py conserva su implementación original y todavía mezcla creación e inserción; no debe confundirse con la nueva herramienta de carga.

Datos de prueba y consultas

7. BD/cargar_tablas.py

Se creó una función cargar_tablas() que reutiliza conectar(). Contiene los 20 productos y los 5 clientes de ejemplo de app.py. Consulta cada tabla por separado e inserta ejemplos únicamente si esa tabla está vacía. Devuelve las cantidades insertadas.

Ambas cargas se realizan en una transacción con BEGIN IMMEDIATE y la conexión se cierra al finalizar. Requiere tablas existentes, conserva registros actuales y no completa cargas parciales. No crea las tablas.

8. BD/cargar_datos.py

Se creó como punto de entrada adicional que llama a cargar_tablas(), evitando repetir la lógica. Incluye el comentario: **SOLO PARA PRUEBAS: estos registros son ejemplos, no datos reales de la tienda.** Al ejecutarse, informa cuántos productos y clientes se insertaron.

Los datos de prueba no desaparecen automáticamente: permanecen en SQLite hasta eliminarlos. Si se vacía una tabla y se vuelve a ejecutar la carga, los ejemplos se insertan otra vez. Importar estos dos módulos no ejecuta la carga.

9. BACKEND/conexionsql.py

Se corrigieron seis problemas: columna cantidad inexistente, resultados guardados como clientes dentro de productos(), intento de cerrar una función con productos.close(), devolución de la función en lugar de las filas, uso de cliente al recorrer productos e impresión de campos incorrectos. También se eliminó la sustitución de los nombres de las funciones por sus resultados.

Ahora clientes() utiliza la conexión compartida y devuelve diccionarios ordenados por ID. productos() delega en persistencia.leer_productos(). Las impresiones se ejecutan únicamente bajo la guarda principal; importar el módulo no consulta la base. Se conservó la lectura de clientes existente sin agregar su CRUD.

10. Otros hallazgos pendientes

En validaciones.py, validar_float() permite que "." llegue a float(), provocando una excepción. Algunas funciones permiten entradas vacías pese a su documentación y las validaciones alfabéticas usan rangos ASCII que excluyen tildes y ñ. Los TDA aún no se integraron con los datos del negocio; también hay funciones heredadas de ejercicios de alumnos en tad_listas.py.

Verificaciones y continuación

11. Verificaciones realizadas durante la sesión

Servidor: se inició con un puerto temporal, se obtuvo HTTP 200 y se comprobó el cierre. Se verificó que importar main no lo inicia.

Lectura real: el usuario ejecutó leer_productos() desde PowerShell y confirmó la devolución de registros como diccionarios.

Carga inicial: en una base en memoria se comprobaron la inserción de 20 productos y 5 clientes, una segunda ejecución sin duplicados y la carga independiente de una tabla vacía. Se verificó la importación de cargar_datos sin ejecutar la carga.

Consultas corregidas: se comprobó en memoria que importar conexionsql no abre conexiones, que clientes y productos devuelven los campos correctos y que las funciones pueden llamarse repetidamente.

Límites de validación: las pruebas de carga y consultas no escribieron en tablas.db. No se ejecutó una prueba integral de la tienda ni se verificó todo el CRUD de productos; esos resultados no se dan por garantizados en este informe.

12. Comandos de uso desde TIENDA_TP

Iniciar el servidor y abrir http://127.0.0.1:8000. Detener con Ctrl+C:

```
python main.py
```

Consultar productos sin generar archivos de caché:

```
python -B -c "from BD.persistencia import leer_productos; print(leer_productos())"
```

Cargar ejemplos solo en tablas vacías (este comando sí puede escribir datos):

```
python -B -m BD.cargar_datos
```

Mostrar clientes y productos mediante el módulo corregido:

```
python -B -m BACKEND.conexionsql
```

13. Próximos pasos acordados

1. Revisar las primitivas de la lista para trabajar con diccionarios por id_producto.
2. Crear BACKEND/inventario.py y cargar los productos persistidos en el TDA Lista.
3. Integrar el inventario al inicio del servidor y ofrecer GET /api/productos.
4. Crear la interfaz con identidad visual y fetch; completar la gestión de productos.
5. Integrar compras y reversión con pila, esperas con cola y recomendaciones con BST.
6. Reconstruir las estructuras al reiniciar, probar los flujos completos y preparar la defensa oral.